

# Smart Home Control Agent

## 1. Project Title

### Smart Home Control Agent using AI

## 2. Project Overview

The Smart Home Control Agent is an AI-powered web application that enables users to manage and control smart home devices through natural language conversations. Instead of manually operating devices, users can simply type commands such as "Turn on the Living Room Light" or "Set Bedroom AC temperature to 24°C," and the AI interprets the request and performs the corresponding action.

The application also supports user authentication, device management, automation scheduling, and AI-powered chat, providing a secure and intelligent smart home experience.

## 3. Objectives

- Develop an AI-powered smart home assistant.
- Provide secure user authentication using JWT.
- Allow users to control smart home devices through AI chat.
- Support CRUD operations for smart devices.
- Enable users to create and manage automations.
- Provide a responsive and modern web interface.

## 4. Features

### Authentication

- User Registration
- User Login
- JWT Authentication
- Protected Routes
- User Logout

## **Device Management**

- Add Device
- Edit Device
- Delete Device
- View Devices
- Device Status (ON/OFF)

## **AI Smart Home Assistant**

- Natural Language Commands
- Device Control
- Device Status
- List Devices
- Count Devices
- AI Smart Home Responses
- Smart Home Command Parsing using Groq LLM

## **Automation Management**

- Create Automation
- Edit Automation
- Delete Automation
- Enable Automation
- Disable Automation
- Scheduler Service

## **Additional Features**

- Search Chat
- Export Chat (PDF)
- Export Chat (TXT)
- Clear Chat
- Quick Action Buttons
- Responsive Dashboard

## **5. Technology Stack**

### **Frontend**

- React.js
- React Router DOM
- CSS
- JavaScript

- Fetch API

## Backend

- FastAPI
- Python

## Database

- SQLite
- SQLAlchemy ORM

## Authentication

- JWT (JSON Web Token)
- OAuth2 Password Bearer

## AI

- Groq API
- Llama 3.3 70B Versatile

## Deployment

- GitHub
- Render

## 6. System Architecture

Frontend (React)

|



FastAPI Backend

|

┌──────────────────┐

| Authentication |

| Device APIs |

| Automation |



## 7. Project Modules

### Module 1 – Authentication

Handles:

- User Registration
- Login
- JWT Token Generation
- Protected APIs

### Module 2 – Device Management

Allows users to:

- Create Devices
- Update Devices
- Delete Devices
- View Devices

### Module 3 – AI Assistant

Receives natural language input and converts it into structured smart home actions.

Examples:

- Turn ON Living Room Light
- Turn OFF Bedroom Fan
- Set AC Temperature
- List Devices

- Count Devices

## **Module 4 – Automation**

Supports:

- Daily Scheduling
- Enable Automation
- Disable Automation
- Update Schedule
- Delete Automation

## **Module 5 – Chat Management**

Features:

- Chat Interface
- Search Conversation
- Delete Message
- Export Chat
- Clear Chat

## **8. API Endpoints**

### **Authentication**

- POST /auth/register
- POST /auth/login

### **Devices**

- GET /devices
- POST /devices
- PUT /devices/{id}
- DELETE /devices/{id}

### **Automations**

- GET /automations
- POST /automations
- PUT /automations/{id}
- DELETE /automations/{id}

## **AI Chat**

- POST /chat

## **9. Database Tables**

### **Users**

- id
- username
- email
- password

### **Devices**

- id
- name
- type
- room
- status
- owner\_id

### **Automations**

- id
- name
- condition
- action
- enabled
- owner\_id

### **Chat History**

- id
- user\_message
- ai\_response
- created\_at
- owner\_id

## **10. Security Features**

- JWT Authentication
- Password Hashing

- Protected API Endpoints
- User Isolation using owner\_id
- CORS Configuration

## 11. Project Workflow

1. User registers.
2. User logs in.
3. JWT token is generated.
4. User accesses dashboard.
5. User manages smart devices.
6. User creates automations.
7. User chats with AI.
8. AI processes command.
9. Backend updates database.
10. Response is displayed to the user.

## 12. Folder Structure

smart-home-control-agent

backend/

```
|
├── app/
│   ├── api/
│   ├── core/
│   ├── database/
│   ├── models/
│   ├── schemas/
│   ├── services/
│   ├── prompts/
│   └── main.py
└── requirements.txt
```

frontend/

```
|
├── src/
│   ├── api/
│   ├── components/
│   ├── context/
│   ├── hooks/
│   ├── layouts/
│   ├── pages/
│   ├── routes/
│   └── App.jsx
```

### **13. Future Enhancements**

- Voice Commands
- Real IoT Device Integration
- MQTT Support
- Alexa Integration
- Google Home Integration
- Push Notifications
- Mobile Application
- Cloud Database (PostgreSQL)
- Multi-room Device Grouping
- AI Conversation History per User

### **14. Challenges Faced**

- JWT Authentication Integration
- AI Prompt Engineering
- Smart Home Command Parsing
- Device State Synchronization
- Frontend and Backend Integration
- Responsive UI Design
- Chat History Management

### **15. Conclusion**

The Smart Home Control Agent successfully demonstrates how Artificial Intelligence can simplify smart home management using natural language. The project combines React, FastAPI, SQLite, JWT Authentication, and Groq Llama 3.3 to provide an intelligent, secure, and user-friendly platform for controlling smart home devices and automations. The modular architecture makes the system scalable and suitable for future enhancements such as IoT integration, voice assistants, and cloud deployment.